



Multi-Agent & Kết Nối Hệ Thống

AICB-P1 · Ngày 9 · MCP, A2A & LangGraph

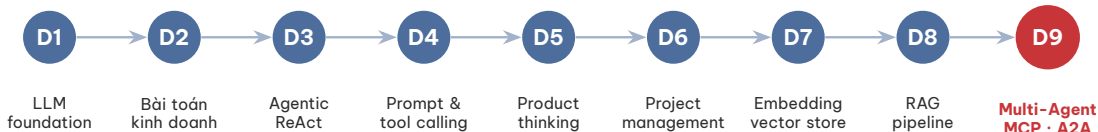
Tên Giảng Viên

VinUniversity · Phase 1 · Tuần 2 · 2026

HÃY SUY NGHĨ...

“Bạn có 1 agent rất giỏi. Nhưng bài toán đã quá lớn cho 1 agent. Làm thế nào để hệ thống vẫn rõ vai trò, dễ kiểm soát, và dễ mở rộng?”

Giữ câu hỏi này trong đầu khi học bài hôm nay



Vị trí hôm nay

Day 08 đã dạy cách **lấy đúng thông tin**. Day 09 hỏi câu tiếp theo: khi bài toán lớn hơn một agent, ta **tổ chức hệ thống** như thế nào?

1. Giới hạn của single-agent
2. Mental model: tư duy hệ thống
3. Multi-agent patterns
4. Supervisor-worker deep dive
5. MCP – chuẩn kết nối tool
6. A2A – giao tiếp giữa agents
7. Orchestration với LangGraph
8. Observability & debugging
9. Cost, latency & reliability
10. Kế hoạch học tập Ngày 9
11. Lab 9 + deliverable

- Giải thích được vì sao **single-agent** bắt đầu quá tải khi bài toán cần nhiều vai trò và nhiều nguồn lực
- Phân biệt được các pattern **supervisor-worker**, **pipeline**, **debate**, và **hierarchical** – và biết chọn đúng pattern
- Hiểu **MCP** là chuẩn nối agent với tool / service bên ngoài, và **A2A** là cách agents giao việc cho nhau với message contract rõ
- Dùng **LangGraph** để hình dung graph, state, và conditional routing trong hệ multi-agent
- Thiết kế được **trace & observability** để debug và cải thiện hệ thống
- Nâng cấp artifact Day 08 thành hệ thống **Supervisor + Workers** có trace rõ ràng

Artifact pack cần nộp

Bản nâng cấp từ Day 08 gồm supervisor, 2–3 workers, 1 kết nối tool qua MCP, và trace log cho toàn bộ luồng phối hợp

- 1 supervisor nhận task, route đúng worker, và tổng hợp kết quả
- 2–3 worker chuyên vai trò như retrieval, tool use, synthesis
- 1 kết nối external capability qua **MCP**
- 1 trace dễ đọc để giải thích agent nào đã làm gì và khi nào

Từ Single-Agent Sang Multi-Agent

Day 08 giúp agent biết retrieve và trả lời grounded; Day 09 trả lời câu hỏi khi nào một agent không còn đủ để gánh toàn bộ bài toán

Day 08 đã làm được

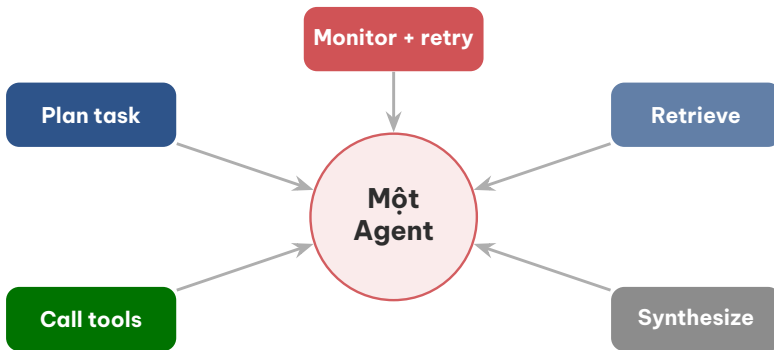
- retrieve đúng tài liệu hơn
- rerank hoặc lọc context tốt hơn
- generate câu trả lời grounded hơn

Nhưng khi hệ thống lớn lên

- phải phân tích task trước khi retrieve
- phải gọi thêm tool ngoài
- phải chia việc và tổng hợp nhiều kết quả
- phải theo dõi trace để debug

Thông điệp mở bài

Day 09 không phủ định Day 08. Nó là bước tiếp theo: **biến một agent có RAG thành một hệ thống có vai trò, có phối hợp, và có điểm mở rộng rõ ràng.**



Một nơi phải làm quá nhiều việc sẽ khó tối ưu, khó debug, và khó scale.

Lưu ý: Câu hỏi đúng không còn là “agent có đủ thông minh không?” mà là “ta có đang ép một agent gánh quá nhiều vai trò không?”

1. Context bottleneck

Một agent phải giữ quá nhiều mục tiêu, tool outputs, evidence, và state trong cùng một lần suy luận. Context window có **giới hạn cứng**.

2. Specialization trade-off

Agent càng ôm nhiều vai, prompt càng dài và khó ổn định. Giải đều mọi thứ thường đồng nghĩa với không thật sự giỏi vai nào.

3. Parallelism hạn chế

Một agent thường chạy tuần tự. Khi có nhiều việc độc lập, hệ thống vẫn phải chờ từng bước nối nhau – tăng latency không cần thiết.

4. Reliability yếu

Nếu agent chọn sai tool hoặc hiểu sai task ở đầu luồng, toàn bộ hệ thống dễ đi lệch theo. Không có **isolation** để khoanh vùng lỗi.

Kịch bản thực tế

- Agent nhận task: phân tích hợp đồng 80 trang + tra cứu luật + tóm tắt rủi ro
- Tool call trả về **12.000 tokens**
- Chat history đã chiếm **6.000 tokens**
- Prompt gốc: **3.000 tokens**
- Còn lại cho reasoning: $\approx$ **3.000 tokens**

Dấu hiệu nhận biết

- câu trả lời thiếu thông tin ở giữa document
- agent lặp lại bước đã làm
- reasoning ngắn bất thường ở bước cuối
- tool call với empty context

Lưu ý: Agent bắt đầu “quên” phần đầu tài liệu khi xử lý phần cuối – **lost-in-the-middle problem**.

- ✓ **Task có nhiều bước vai trò khác nhau:** plan, retrieve, tool use, tổng hợp
- ✓ **Có thể chia việc độc lập:** 2 worker làm song song vẫn hợp lý
- ✓ **Cần debug rõ ai làm sai:** route sai, retrieve sai, hay synthesis sai
- ✓ **Cần mở rộng dần:** thêm 1 worker mới mà không viết lại cả prompt gốc
- ✓ **Context window một agent không đủ:** tài liệu lớn, nhiều tool output
- ☐ **Đừng dùng multi-agent chỉ vì thấy “ngầu”.** Nếu 1 workflow đơn giản đã đủ, giữ đơn giản sẽ rẻ và ổn định hơn

Mental Model: Tư Duy Hệ Thống Trước Khi Thiết Kế

Trước khi chọn pattern hay tool, cần hình thành tư duy đúng về cách chia trách nhiệm trong một hệ thống phức tạp

Tư duy cũ

- Làm thế nào để agent thông minh hơn?
- Prompt nào khiến agent làm được nhiều hơn?
- Thêm nhiều tool cho một agent để đủ sức

Lưu ý: Tư duy này dẫn đến “**god agent**” – một agent làm hết nhưng không ai hiểu nó đang làm gì.

Tư duy hệ thống

- Task này gồm bao nhiêu loại trách nhiệm khác nhau?
- Ai cần biết gì, khi nào?
- Lỗi cần được khoanh vùng ở đâu?
- Điểm nào cần human oversight?

Kết quả

Hệ thống clear về vai trò, dễ test từng phần, và dễ cải thiện dần.

Câu hỏi 1 – Chia trách nhiệm ở đâu?

Task nào cần reasoning? Task nào cần data fetching? Task nào chỉ cần format?

Câu hỏi 2 – Thông tin đi theo con đường nào?

Agent nào cần biết gì?
Ai cần đầu ra của ai trước tiên?

Câu hỏi 3 – Lỗi ở đâu là ít tổn hại nhất?

Thiết kế để lỗi tại worker không làm hỏng toàn bộ hệ thống.

Nguyên tắc

Thiết kế tốt giúp **lỗi có địa chỉ rõ ràng** thay vì lan ra cả hệ thống.

MINI-QUEST 1

20 phút · cá nhân hoặc cặp · làm trực tiếp trên máy

Phần A – Điều tra (8')

Chọn công cụ bạn dùng hằng ngày: **Claude Code, Codex, Antigravity, OpenCode, Cursor...**

Tìm **bằng chứng ngay trong máy**, không đoán:

- có cơ chế **gọi agent con** không? gọi là gì?
- agent con có **context riêng** hay dùng chung?
- chạy **song song** được mấy cái?
- có **giới hạn tool** cho từng agent được không?
- tool ngoài nối vào bằng đường nào?

Phần B – Tự tạo 1 agent (10')

Viết một agent con cho *chính công cụ bạn đang dùng*, rồi **chạy thử một lần thật**.

Ba ràng buộc:

- **một vai trò hẹp** – không phải trợ lý đa năng
- **mô tả rõ khi nào được gọi** – đây chính là tín hiệu route cho supervisor
- **cắt bớt tool** nó được phép dùng

Mỗi sẵn: repo môn học đã có `.claude/agents/*.md` và `.codex/agents/*.toml` – **cùng 5 agent, hai schema khác nhau**. Mở đọc trước khi tự viết. · **2' cuối:** 2 bạn chia sẻ kết luận + bằng chứng.

Thứ bạn thấy trong công cụ	Khái niệm Day 09	Bằng chứng trong repo môn học
Vòng lặp chính nhận yêu cầu rồi quyết định gọi ai .claude/agents/*.md, .codex/agents/*.toml	Supervisor Định nghĩa Worker	phiên chat gốc của Claude Code / Codex 5 agent: slide-reviewer, lab-smoke-tester, vn-content-reviewer...
Dòng description: "use when..." Dòng tools:	Tín hiệu routing Trust boundary / least privilege	supervisor đọc đúng dòng này để chọn worker vn-content-reviewer chỉ có Read, Grep, Glob – không có Write , nên không thể sửa deck
Agent con có context window riêng max_threads = 6 max_depth = 1	Chống context bottleneck Parallelism Chặn hierarchical	lý do thật sự để tách worker – không phải vì “ngẫu” .codex/config.toml worker không được để worker → tránh đệ quy vô hạn

Công cụ bạn dùng hằng ngày chính là ví dụ Day 09 gần nhất

Multi-Agent Patterns

Có nhiều cách chia hệ thống thành nhiều agent; điều quan trọng là chọn pattern giúp giải quyết đúng loại phức tạp, không tạo thêm phức tạp giả

Supervisor-Worker

1 supervisor điều phối nhiều worker chuyên biệt.

Mạnh ở: routing rõ, dễ kiểm soát, dễ trace

Pipeline

Agent A xong rồi mới chuyển output cho B.

Mạnh ở: flow ổn định, tuyến tính, dễ test

Debate

Nhiều agent cùng giải một bài toán rồi vote hoặc synthesize.

Mạnh ở: phản biện và giảm blind spot

Hierarchical

Supervisor lồng supervisor cho nhiều tầng hệ thống.

Mạnh ở: mở rộng tốt ở enterprise scale

Lựa chọn cho Day 09: đi sâu vào **supervisor-worker** – dễ dạy, dễ build lab, dễ nối từ artifact Day 08.

Pattern	Dùng khi nào	Điểm mạnh	Cảnh báo
Supervisor-worker	Task cần route tới đúng vai trò	dễ kiểm soát, dễ trace	supervisor có thể thành bottleneck nếu làm quá nhiều
Pipeline	Các bước gần như cố định	dễ hiểu, dễ test theo bước	kém linh hoạt khi flow đổi động
Debate	Cần nhiều góc nhìn cho cùng một bài toán	giảm blind spot, tăng phản biện	tốn cost và khó tổng hợp
Hierarchical	Nhiều nhóm task và nhiều tầng quản trị	mở rộng tốt ở quy mô lớn	thiết kế và debugging phức tạp

Đừng để 4 pattern ngang nhau trong đầu người học

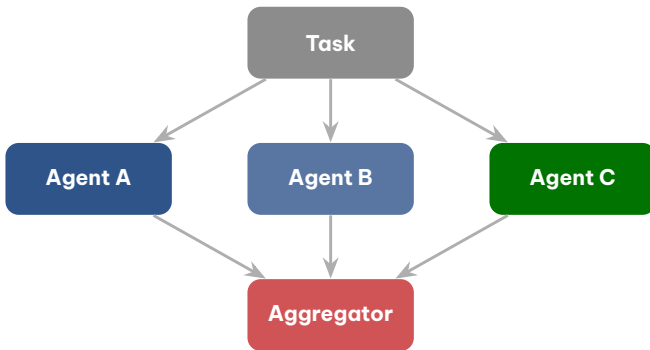


Phù hợp nhất khi

- flow gần như cố định
- mỗi bước cần output của bước trước
- dễ test từng agent riêng biệt

Hạn chế

- latency cộng dồn ở mỗi bước
- khó xử lý khi flow cần rẽ nhánh
- retry một bước ảnh hưởng toàn chuỗi



Dùng khi nào

Khi bài toán có **nhiều góc nhìn hợp lệ**, khi rủi ro sai cao, hoặc khi cần **kiểm tra chéo** trước một quyết định quan trọng.

Lý do sơ phạm

- học viên dễ nhìn ra vai trò
- dễ nối với use case thật
- dễ giải thích logic route
- dễ nâng cấp từ artifact Day 08

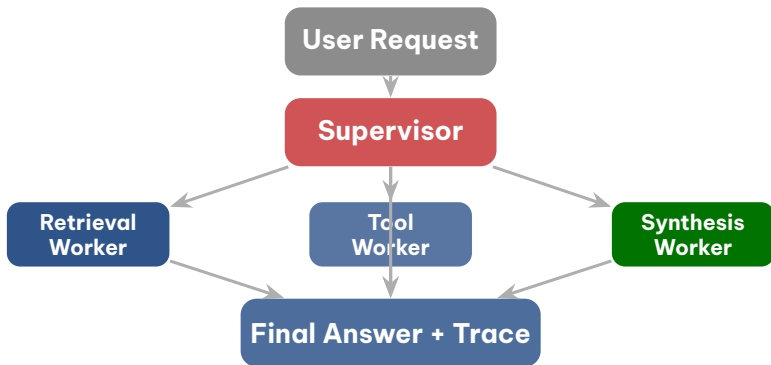
Lý do triển khai

- bắt đầu từ 2-3 worker là đủ
- dễ cắm thêm MCP tool worker
- trace và testing rõ hơn
- supervisor thường chỉ cần một LLM call nhỏ

Lưu ý: Nếu Day 09 ôm nhiều pattern ngang nhau, người học sẽ nhớ tên pattern nhưng không biết ngày mai nên build theo pattern nào.

Supervisor-Worker: Deep Dive

Thay vì ép một agent làm hết, ta cho một supervisor phân việc và nhiều worker làm phần việc hẹp, dễ kiểm soát hơn



Khung nghĩ đúng

Supervisor không cần “thông minh hơn tất cả”. Vai trò chính là **chia việc đúng, gọi đúng worker, và gom đầu ra thành kết quả mạch lạc**.

Supervisor

- phân tích yêu cầu ban đầu
- quyết định worker nào nên tham gia
- theo dõi trạng thái và retry nếu cần
- tổng hợp đầu ra cuối cùng
- biết khi nào cần human review

Worker

- xử lý một năng lực hẹp
- nhận input rõ ràng, trả output rõ ràng
- càng stateless càng dễ test
- thất bại cục bộ không làm hỏng cả kiến trúc
- có thể được thay thế mà không ảnh hưởng supervisor

Rule of thumb

Supervisor giữ **decision flow**; worker giữ **domain skill**. Đừng để một worker vừa làm việc hẹp vừa bí mật điều phối cả hệ thống.

Specialized

Một worker nên có một năng lực chính: retrieve, gọi tool, tóm tắt, kiểm tra policy...

Stateless ưu tiên

Nếu có thể, worker chỉ cần input hiện tại thay vì ôm cả lịch sử hệ thống.

Testable

Có input / output rõ để test độc lập trước khi cắm vào supervisor.

Lưu ý: Worker mơ hồ thường làm debugging cực khó: không biết lỗi do route sai, prompt sai, hay contract đầu vào chưa đủ rõ.

God Supervisor

Supervisor làm quá nhiều: plan, retrieve, synthesize, monitor. Nó trở thành **single-agent được đổi tên**.

Implicit State

State bị truyền qua biến toàn cục hoặc side effect. Không ai biết hệ đang ở bước nào.

Chatty Workers

Workers liên tục gọi ngược lại supervisor để hỏi thêm thông tin. Message overhead tăng rất nhanh.

No Fallback

Worker gặp lỗi và không trả về gì. Supervisor chờ mãi không thấy đầu ra để tổng hợp.

Shared state

- dễ xem toàn cảnh
- tiện cho graph orchestration (LangGraph)
- nhưng dễ bị “đụng tay” lẫn nhau nếu không có kỷ luật
- cần schema rõ: ai được đọc gì, ai được ghi gì

Message passing

- contract rõ hơn giữa các agents
- ít coupling hơn
- nhưng phải thiết kế message format cẩn thận
- cần validation ở mỗi điểm nhận

Cách dạy thực dụng

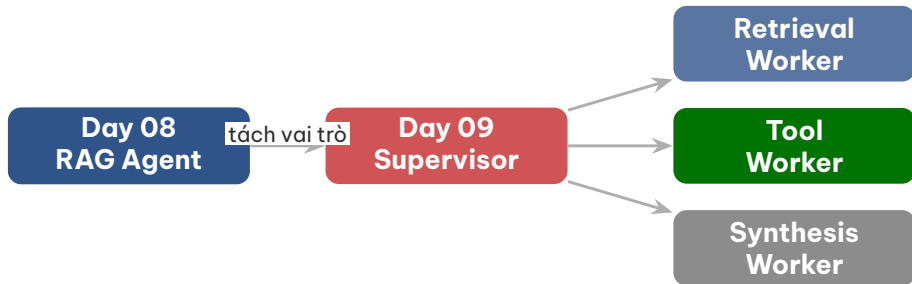
Trong Day 09, học viên chỉ cần nhớ: **shared state giúp điều phối**, còn **message contract giúp giao tiếp không nhập nhằng**.

```
class Day09State(TypedDict):  
    task: str                # task gốc từ user  
    plan: list[str]          # worker cần gọi  
    worker_results: dict     # output từ worker  
    status: str              # pending/running/done  
    final_answer: str        # output gộp cuối  
    trace: list[dict]        # log có timestamp  
    error: Optional[str]
```

Tại sao trace là trường bắt buộc?

Không có trace trong state, sau khi hệ chạy xong **không ai biết agent đã đi theo con đường nào** để ra kết quả đó.

Quy tắc: trace là list, luôn **append**, không bao giờ overwrite.



Thông điệp lab

Day 09 không bắt đầu từ số 0. Ta lấy năng lực retrieve và answer của Day 08 rồi **chia vai trò thành các worker** để hệ thống rõ ràng hơn.

MINI-QUEST 2

20 phút · 5' cá nhân + 8' nhóm + 7' chữa chung

```
def supervisor_node(state: AgentState) -> AgentState:
    decision = llm.invoke(
        SUPERVISOR_PROMPT.format(task=state["task"])
    )

    docs = vector_store.search(state["task"], k=20)
    state["retrieval_result"] = docs

    state["trace"] = [f"[supervisor] {decision}"]

    result = tool_worker(state)
    state["final_answer"] = result["text"]

    return state
```

Câu hỏi cho nhóm

Đoạn code chạy được, không crash, nhưng vi phạm **ít nhất 4 nguyên tắc** đã học sáng nay.

1. Tìm đủ 4 lỗi
2. Gọi tên anti-pattern tương ứng cho từng lỗi
3. Với mỗi lỗi, viết 1 dòng mô tả cách sửa
4. Lỗi nào sẽ khiến việc debug khó nhất? Vì sao?

Lỗi trong code	Anti-pattern	Cách sửa
Supervisor tự gọi <code>vector_store.search</code>	God Supervisor	để Retrieval Worker làm; supervisor chỉ set <code>need_retrieval</code> rồi route
<code>state["trace"] = [...]</code> ghi đè	Mất observability	<code>append: "trace": state["trace"] + [entry]</code>
Gọi thẳng <code>tool_worker(state)</code> trong node	Routing bị chôn trong code	dùng conditional edge của graph; node chỉ ra quyết định, không tự gọi worker
Không kiểm tra kết quả worker, không <code>try/except</code> , không set status	No Fallback	validate output, set <code>status=error</code> , có retry và đường thoát khi worker fail
<i>Bonus: node mutate state tại chỗ rồi return chính nó → nên trả về dict mới để state có thể replay / persist.</i>		

Đáp án Mini-Quest 2

MCP – Model Context Protocol

Nếu supervisor-worker là cách chia người làm việc, thì MCP là cách agent cắm được vào năng lực bên ngoài mà không phải custom từng tích hợp từ đầu

Trước MCP – vấn đề thực tế

- mỗi tool cần một adapter riêng
- thay đổi API của tool = viết lại code tích hợp
- mỗi framework gọi tool theo cách khác nhau
- không có chuẩn chung để agent biết tool làm gì

MCP giải quyết

Một chuẩn giao tiếp duy nhất giữa agent và tool. Agent biết cách **khám phá** các capability mà không cần hard-code từng tích hợp.

Lưu ý: Điều quan trọng với người học là hiểu **vì sao MCP giúp mở rộng hệ thống**, không phải học thuộc protocol spec trong buổi đầu tiên.

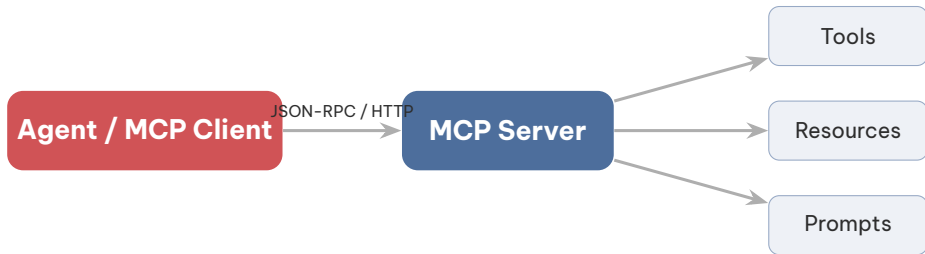
- **MCP** là một chuẩn để agent kết nối với external capabilities.
- Thay vì mỗi tool có một kiểu tích hợp riêng, agent có thể nói chuyện với một **MCP server**.
- MCP server công bố các thứ như **tools**, **resources**, và **prompts**.
- Agent có thể **list**, **describe**, và **invoke** các capability đó theo chuẩn chung.

Analogy dễ nhớ

Supervisor-worker nói về **vai trò**.

MCP nói về **ổ cắm chuẩn** để agent dùng tài nguyên bên ngoài.

Giống **USB**: mọi thiết bị cùng dùng một chuẩn kết nối.



Điểm cốt lõi

Agent không cần biết chi tiết từng hệ thống phía sau. Nó chỉ cần hiểu **MCP surface** mà server công bố.

Tools

Hành động hoặc thao tác

Ví dụ: search, query API, tạo ticket, gọi webhook

Resources

Tài nguyên để đọc

Ví dụ: file, schema, catalog, config, DB

Prompts

Prompt dùng lại

Giúp chuẩn hóa cách gọi năng lực và giảm lỗi prompt

Lưu ý: Không phải MCP server nào cũng phải có đủ cả ba. Điều quan trọng là agent có một **cách nhìn nhất quán** về các capability được công bố.

Luồng discovery

1. Agent kết nối tới MCP server
2. Gọi `tools/list` để lấy danh sách tool
3. Mỗi tool trả về: `name`, `description`, `inputSchema`
4. Agent đọc schema, quyết định tool nào phù hợp
5. Gọi `tools/call` với đúng parameters
6. MCP server thực thi và trả kết quả

Tại sao quan trọng?

Agent không cần được lập trình sẵn biết tool “X” tồn tại. Nó có thể **khám phá khi chạy** và tự điều chỉnh theo tool nào có sẵn.

Kịch bản: Customer Support Agent

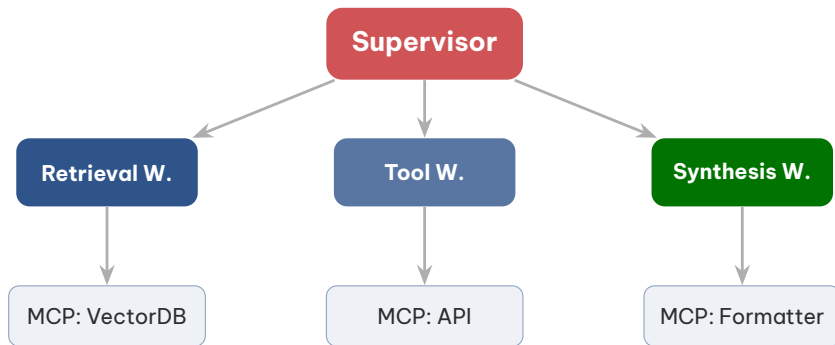
- Tool Worker cần tra policy mới nhất
- Gọi MCP server “knowledge-base”
- Server expose tool: `search_policy`, `get_faq`
- Worker gọi `search_policy(query, date_after)`
- Kết quả trả về JSON chuẩn kèm source

Lợi ích trực tiếp

- Team cập nhật policy → chỉ cần update MCP server
- Supervisor và workers không cần sửa khi tool thay đổi
- Thêm tool mới = thêm endpoint vào MCP server
- Dễ audit ai đã gọi tool gì và khi nào

Thông điệp chốt section

MCP quan trọng vì nó tạo **ecosystem effect**: agent dùng được nhiều năng lực hơn mà không phải mỗi lần đều tích hợp lại từ đầu.



Điểm cần nhớ

MCP là **lớp giữa** worker và capability thực. Worker chỉ cần biết MCP surface, không cần biết hệ thống phía sau là gì.

A2A – Agent to Agent Communication

MCP giúp agent nói chuyện với tool; A2A giúp agent nói chuyện với agent khác theo cách rõ nhiệm vụ, rõ bối cảnh, và rõ đầu ra mong đợi

MCP

- agent nói chuyện với **tool / capability**
- mục tiêu là **kết nối năng lực bên ngoài**
- trọng tâm là surface chuẩn
- tool **không có agency** – chỉ thực thi

A2A

- agent nói chuyện với **agent khác**
- mục tiêu là **chia việc và đồng bộ**
- trọng tâm là message contract rõ ràng
- agent phía kia **có thể ra quyết định**

Lưu ý: Cùng là “gọi ra ngoài”, nhưng **MCP trả lời câu hỏi agent lấy năng lực ở đâu**, còn **A2A trả lời câu hỏi agent giao việc cho ai**.

Không có contract rõ

- supervisor gọi worker với: “Hãy tìm policy liên quan”
- worker không biết context là gì
- worker trả về 10 kết quả không được lọc
- supervisor không biết kết quả nào dùng được
- lỗi lộ ra ở phần tổng hợp, nhưng gốc là ở phần gọi

Với contract rõ

Supervisor gọi worker với đầy đủ **task + context + expected format**. Worker biết chính xác cần làm gì và trả về theo schema đã thống nhất.

Task

Agent kia cần làm gì?
Ví dụ: tìm 3 chunk pol-
icy phù hợp nhất

Context

Những gì worker cần
biết để làm đúng?
query, constraints,
user role, state

Expected output

Trả về theo format
nào?
list chunks, score,
rationale, error

Ví dụ:

task = "retrieve evidence"

context = "user hỏi về refund policy, ưu tiên tài liệu sau 2025"

expected_output = "top 3 chunks + source + confidence"

Thiếu context

- worker lấy kết quả không phù hợp
- phải gọi lại nhiều lần
- debugging rất khó

Quá nhiều context

- tốn token không cần thiết
- worker xử lý chậm
- khó maintain khi schema thay đổi

Nguyên tắc “need to know”

Worker chỉ nhận context mà nó **thực sự cần** để hoàn thành task của mình. Không thêm, không bớt.

Khi không chắc → bắt đầu với **ít hơn** và thêm khi cần.

Sync

- đơn giản để hiểu và debug
- phù hợp khi supervisor cần kết quả ngay để đi bước tiếp
- nhưng dễ tăng latency toàn luồng
- **bắt đầu ở đây cho Day 09**

Async

- hợp khi nhiều worker chạy song song
- tận dụng được concurrency
- nhưng cần quản lý trạng thái và timeout tốt hơn
- mở rộng khi đã nắm sync tốt

Cách giảng đơn giản

Sync dễ dạy cho vòng đầu. Async chỉ cần giới thiệu như một hướng mở rộng khi nhiều worker có thể chạy độc lập.

- ☒ **Biết rõ ai được gọi ai:** không phải agent nào cũng được chạm mọi capability
- ☒ **Biết dữ liệu nào được truyền đi:** tránh đẩy thừa PII hoặc state nhạy cảm
- ☒ **Biết output nào cần xác thực lại:** đặc biệt khi worker chạm tool ngoài
- ☒ **Validate đầu ra trước khi dùng:** worker hoàn toàn có thể trả về schema sai
- ☐ **Đừng giả định mọi agent đều đáng tin như nhau.** Hệ nhiều agent vẫn cần trust boundary

Orchestration Với LangGraph

Sau khi hiểu vai trò và giao tiếp, ta cần một cách biểu diễn luồng chạy rõ ràng; LangGraph là cách rất trực quan để làm điều đó

Không có framework

- routing logic nằm trong prompt điều phối dài
- khó biết hệ đang ở bước nào
- thêm một nhánh mới = sửa toàn bộ prompt
- debug bằng `print()` và hy vọng

Với LangGraph

Routing trở thành **code tường minh**. Graph có thể visualize. State có schema. Human-in-the-loop có điểm rõ ràng.

Lưu ý: Nếu không có orchestration rõ ràng, routing logic thường bị chôn trong prompt và trở nên rất khó debug.

- Biến hệ multi-agent thành **graph** gồm nodes, edges, và state.
- Tách rõ **node nào làm việc gì** và **khi nào route sang node nào**.
- Giúp hệ thống bớt phụ thuộc vào một prompt điều phối khổng lồ.
- Hỗ trợ **persistence**: state có thể được lưu và chạy tiếp.
- Hỗ trợ **human-in-the-loop** tại bất kỳ điểm nào.

Khung nhớ nhanh

node = ai làm
edge = đi đâu tiếp
state = hệ đang biết gì

Ba khái niệm này là toàn bộ LangGraph bạn cần cho Day 09.

Node

- là một hàm Python nhận state và trả state mới
- tương ứng với một agent hoặc một bước xử lý
- có thể là supervisor, worker, hoặc human review

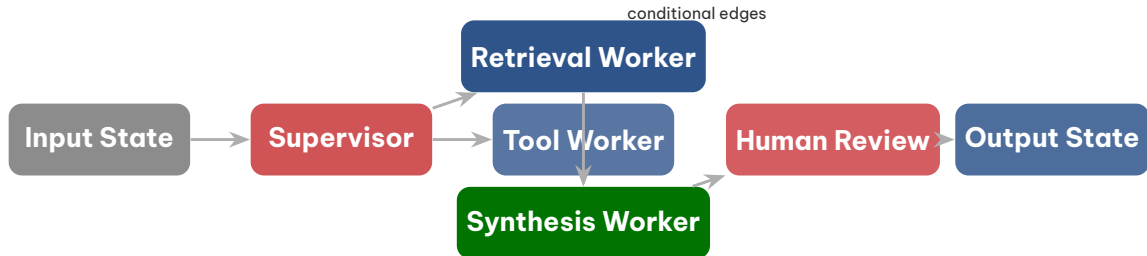
State

Là TypedDict hoặc Pydantic model được truyền qua mỗi node. Mỗi node có thể đọc toàn bộ state và ghi vào các trường được phép.

State là “**bộ nhớ**” của cả graph.

Edge

- **Unconditional edge:** luôn đi từ A sang B
- **Conditional edge:** hàm trả về tên node tiếp theo dựa trên state



Điểm cần nhớ

LangGraph làm rõ **route quyết định ở đâu, state đi qua đâu, và human can thiệp ở điểm nào.**

```
class AgentState(TypedDict):
    task: str
    need_retrieval: bool
    need_tool: bool
    worker_results: dict
    final_answer: str

def route_to_worker(state: AgentState) -> str:
    if state["need_tool"]:
        return "tool_worker"
    if state["need_retrieval"]:
        return "retrieval_worker"
    return "synthesis_worker"
```

```
graph.add_conditional_edges(
    "supervisor",
    route_to_worker,
    {
        "tool_worker": "tool_worker",
        "retrieval_worker": "retrieval_worker",
        "synthesis_worker": "synthesis_worker",
    },
)
```

Ý chính không nằm ở syntax mà ở chỗ: **routing trở thành logic tường minh**, thay vì ẩn trong một prompt điều phối rất dài.

```
def supervisor_node(state: AgentState) -> AgentState:
    decision = llm.invoke(
        SUPERVISOR_PROMPT.format(task=state["task"])
    )
    return {
        **state,
        "need_retrieval": decision.need_retrieval,
        "need_tool": decision.need_tool,
        "trace": state["trace"] + [
            f"[supervisor] retrieval="
            f"{decision.need_retrieval} "
            f"tool={decision.need_tool}"
        ],
    }
```

Đọc gì từ đoạn này?

- supervisor node **không làm việc của worker** – nó chỉ ra quyết định route
- trả về **dict mới** (**state) thay vì sửa state tại chỗ
- trace được **append**, không overwrite

Nên chèn khi

- task có rủi ro cao (tài chính, y tế, pháp lý)
- confidence score dưới ngưỡng
- tool action có side effect không đảo ngược
- output sẽ đi ra user hoặc stakeholder
- hệ thống không chắc về intent ban đầu

Cách implement trong LangGraph

- thêm node `human_review` vào graph
- node này **interrupt** graph và chờ input
- sau khi human approve → chạy tiếp
- state được giữ nguyên qua interrupt
- log lại quyết định của human trong trace

Khung nghĩ đúng

Multi-agent không đồng nghĩa với full autonomy. Nhiều hệ tốt nhất là hệ biết **khi nào nên dừng để con người quyết định.**

Observability & Debugging Hệ Multi-Agent

Hệ thống nhiều agent khó debug hơn nhiều so với single agent; observability tốt là điều kiện tiên quyết để cải thiện được hệ thống

Nguồn gốc lỗi khó xác định

- Lỗi có thể xuất phát từ: routing sai, context sai, tool fail, synthesis sai
- Lỗi ở bước A có thể chỉ lộ ra ở bước C
- Nhiều agent = nhiều LLM call = nhiều điểm fail tiềm năng

3 câu hỏi observability

1. Agent nào đã chạy, theo thứ tự nào?
2. Input / output tại mỗi bước là gì?
3. Lỗi hay warning nào đã xảy ra?

Lưu ý: Không có trace tốt, debugging multi-agent gần như là **mò mẫm**.

Mỗi entry trong trace nên có

- timestamp — khi nào
- agent_id — ai làm
- action — làm gì (route / call / synthesize / error)
- input_summary — nhận gì (tóm tắt, không full context)
- output_summary — trả về gì
- status — ok | warn | error
- latency_ms — mất bao lâu

```
{  
  "t": "14:03:21",  
  "agent": "supervisor",  
  "action": "route",  
  "decision": "retrieval_worker",  
  "reason": "need_retrieval=true",  
  "status": "ok",  
  "latency_ms": 312  
}
```

- supervisor nhận task gì và route theo tiêu chí nào
- worker nào được gọi và input nó nhận là gì
- worker trả về output gì, confidence hoặc status gì
- supervisor đã tổng hợp ra answer cuối cùng như thế nào
- điểm nào bị retry, timeout, hoặc fallback
- nếu có human review, human đã quyết định gì

Lưu ý: Nếu log chỉ có “agent chạy xong”, học viên sẽ không học được gì về orchestration. Trace tốt phải giúp nhìn thấy **đường đi của quyết định**.

LangSmith

Tích hợp sẵn với LangChain / LangGraph. Trace tự động, visual flow, so sánh runs.

JSON log tự viết

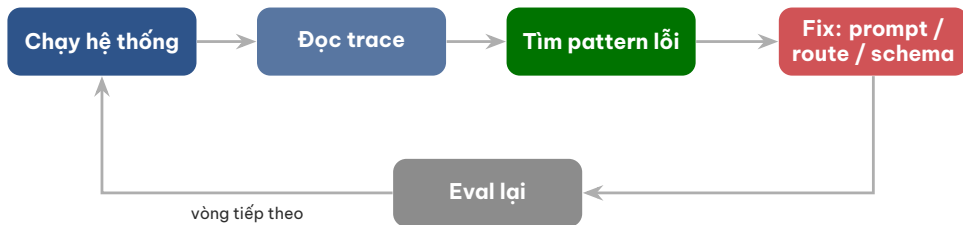
Structured output ghi thẳng vào state. Đơn giản nhất cho Day 09 – dễ đọc, dễ inspect.

OpenTelemetry

Chuẩn mở cho distributed tracing. Phù hợp khi hệ thống lớn hơn và cần dashboard.

Gợi ý cho lab

Bắt đầu với **JSON log tự viết vào state**. Đây là cách học được nhiều nhất về cách hệ hoạt động.



Điểm cần nhớ

Trace không chỉ để debug lỗi hôm nay. Nó là **dữ liệu** để cải thiện routing, worker quality, và message contract theo thời gian.

MINI-QUEST 3

20 phút · 7' đọc trace + 8' nhóm + 5' chữa

```
{ "t": "09:14:02", "agent": "supervisor", "action": "route",  
  "decision": "retrieval_worker", "status": "ok", "latency_ms": 240 }  
{ "t": "09:14:03", "agent": "retrieval_worker", "action": "search",  
  "input": "chính sách hoàn ếtin", "output": "0 chunks",  
  "status": "ok", "latency_ms": 890 }  
{ "t": "09:14:04", "agent": "supervisor", "action": "route",  
  "decision": "synthesis_worker", "reason": "retrieval done",  
  "status": "ok", "latency_ms": 180 }  
{ "t": "09:14:09", "agent": "synthesis_worker", "action": "synthesize",  
  "input": "0 chunks", "output": "Chính sách hoàn ếtin là 30 ngày...",  
  "status": "ok", "latency_ms": 5100 }  
{ "t": "09:14:09", "agent": "supervisor", "action": "finalize", "status": "  
  ok" }
```

Tình huống

Khách hàng báo câu trả lời **sai hoàn toàn** – công ty không hề có chính sách 30 ngày. Nhưng mọi dòng trace đều status: ok.

1. Lỗi **thật sự** xảy ra ở dòng nào?
2. Vì sao trace vẫn báo “ok” ở mọi bước?
3. Trace đang **thiếu trường** nào để phát hiện sớm?
4. Sửa ở đâu: routing, worker, contract, hay state?

Root cause

- Dòng 2: retrieval trả **0 chunks** nhưng vẫn ghi status: ok – “không tìm thấy” bị coi là thành công
- Dòng 3: supervisor route tiếp mà **không kiểm tra chất lượng evidence**
- Dòng 4: synthesis worker nhận 0 chunk vẫn tạo ra câu trả lời → **hallucination**
- Lỗi lộ ra ở cuối luồng nhưng gốc nằm ở dòng 2

Sửa ở 3 chỗ

- **Contract:** expected_output phải cho phép insufficient_evidence; synthesis từ chối khi không đủ evidence
- **Routing:** conditional edge – nếu result_count = 0 thì retry với query khác hoặc chuyển human review
- **Trace:** thêm result_count, top_score; kết quả rồi phải là status: warn, không phải ok

Lưu ý: status: ok chỉ có nghĩa là “bước này không văng exception”. Nó **không** có nghĩa là kết quả dùng được.

Cost, Latency & Reliability

Multi-agent không miễn phí: nhiều agent = nhiều LLM call
= nhiều tiền và nhiều điểm fail; cần tư duy trade-off từ sớm

Chi phí tăng từ đâu?

- Mỗi agent = ít nhất 1 LLM call
- Supervisor thường là một LLM call riêng
- Context truyền qua state có thể rất lớn
- Retry = gấp đôi cost tại điểm đó
- Human review = tăng latency, không tăng cost LLM

Nguyên tắc tối ưu

Supervisor **không cần là model lớn nhất**. Nếu routing logic đơn giản, dùng model nhỏ hơn cho supervisor và giữ model mạnh cho worker cần reasoning sâu.

Tiêu chí	Single Agent	Multi-Agent
Chi phí LLM	thấp hơn (1 call)	cao hơn (nhiều call)
Latency	thấp nếu prompt ngắn	có thể song song; tăng nếu chạy se- rial
Debuggability	khó hơn (logic ẩn)	tốt hơn (có trace rõ)
Specialization	hạn chế	tốt hơn (mỗi worker chuyên biệt)
Scalability	khó scale vai trò	dễ thêm worker mới
Complexity	đơn giản hơn	phức tạp hơn khi setup

Trade-off quan trọng cần hiểu khi thiết kế

Cần thiết kế trước

- Worker có timeout rõ ràng
- Supervisor có retry logic: thử lại bao nhiêu lần?
- Nếu retry cũng fail: fallback là gì?
- Thất bại cục bộ không nên crash toàn bộ hệ thống
- Partial failure: tổng hợp kết quả với những worker đã thành công

Graceful degradation

Hệ thống không cần làm tốt mọi trường hợp. Nó cần **thất bại theo cách kiểm soát được** và báo cáo rõ ràng khi không đủ tự tin.

Kế Hoạch Học Tập Ngày 9

Roadmap rõ ràng giúp học viên biết nên đầu tư thời gian vào đâu và cần nắm vững điều gì trước khi chuyển sang bước tiếp

50'	Lý thuyết: single-agent limits, mental model, 4 patterns
20'	Mini-Quest 1: mở xẻ harness bạn đang dùng
40'	Supervisor-worker deep dive, shared state, anti-patterns
20'	Mini-Quest 2: tìm lỗi trong supervisor node
45'	MCP architecture + A2A message contract
30'	LangGraph: nodes, edges, state, routing code
25'	Observability + cost / reliability trade-offs
20'	Mini-Quest 3: đọc trace, tìm root cause
90'	Lab 9: nâng cấp artifact Day 08

Từ Day 08 (bắt buộc)

- RAG pipeline: query → retrieve → generate
- Tool calling cơ bản
- Cách viết system prompt có cấu trúc
- Artifact Day 08 đang hoạt động

Python foundations (cần có)

- TypedDict và type hints
- Dictionary operations
- Function decorators cơ bản
- async/await nếu đi vào async A2A

Lưu ý: Học viên chưa có artifact Day 08 hoạt động nên dành **30 phút đầu** để fix artifact trước khi bắt đầu lab Day 09.

Cấp 1: Foundation

- Giải thích được 4 giới hạn single-agent
- Vẽ được sơ đồ supervisor-worker
- Phân biệt được MCP và A2A

Cấp 2: Implementation

- Build supervisor + 2 workers với shared state
- Viết message contract cho A2A
- Dùng MCP cho 1 tool worker

Cấp 3: Mastery

- Thiết kế LangGraph có conditional routing
- Trace log đọc được và actionable
- Giải thích trade-off cost / reliability

Misconception	Reality
“Nhiều agent = hệ thống tốt hơn”	Nhiều agent = nhiều phức tạp, chỉ nên dùng khi cần
“Supervisor phải là model lớn nhất”	Supervisor chỉ cần đủ để route đúng
“MCP và A2A là cùng một thứ”	MCP: tool integration; A2A: agent delegation
“Multi-agent tự động giải quyết context problem”	Context vẫn phải được quản lý cẩn thận ở từng worker
“LangGraph chỉ dùng được với LangChain”	LangGraph dùng được với nhiều LLM framework khác

Những nhầm lẫn này hay lộ ra ngay trong Mini-Quest 1

Đọc trước (chuẩn bị)

- LangGraph Quickstart (docs.langchain.com)
- MCP Introduction – modelcontextprotocol.io
- Sumers et al. (2023), CoALA – Section 2–3
- Review lại artifact Day 08 của bản thân

Đọc sau (củng cố)

- LangGraph Multi-Agent Tutorial
- Anthropic – Building Effective Agents (blog)
- MCP Server Examples trên GitHub
- LangSmith Tracing Guide

Cách đọc hiệu quả

Hands-on trước, docs sau. Đọc code example thực tế trước khi đọc architecture spec đầy đủ.

Hands-on 9

Lấy artifact Day 08 rồi chia lại thành supervisor, workers, và 1 điểm kết nối MCP để học viên thấy rõ giá trị của phân vai trong thực tế



Mục tiêu lab

Biến một agent RAG đơn thành một hệ multi-agent nhỏ có route rõ, capability rõ, và trace rõ để dễ giải thích và debug hơn.

1. tách artifact Day 08 thành supervisor + 2-3 workers
2. thiết kế shared state schema với trường trace
3. chọn 1 worker dùng external capability qua MCP
4. viết message contract tối thiểu giữa supervisor và workers
5. trace lại toàn bộ luồng để biết agent nào đã làm gì
6. demo kết quả cuối cùng kèm reasoning flow ở mức quan sát được

Câu hỏi cần trả lời

- RAG agent Day 08 hiện đang làm bao nhiêu việc khác nhau?
- Phần nào có thể tách thành worker riêng?
- Phần nào nên để supervisor quyết định?

Gợi ý tách vai trò

- **Retrieval Worker:** vector search + rerank
- **Tool Worker:** gọi API qua MCP
- **Synthesis Worker:** generate final answer

Kiểm tra trước khi tách

Mỗi phần có thể **test độc lập** không? Nếu không, chưa tách đủ rõ.

Supervisor có thực sự cần **ra quyết định** về phần đó không? Nếu không, tách ra là dư thừa.

```
class Day09State(TypedDict):  
    task: str  
    user_context: dict  
    plan: list[str]  
    retrieval_result: list[dict]  
    tool_result: dict  
    synthesis_draft: str  
    final_answer: str  
    status: str  
    trace: list[dict]  
    error: Optional[str]
```

Nguyên tắc thiết kế

- trace là list – luôn append, không overwrite
- error là Optional để graceful fail
- Worker chỉ ghi vào field của mình
- Supervisor đọc tất cả, ghi plan
- Không để field “không ai biết ai sở hữu”

Chọn một trong các lựa chọn lab

- **Tùy chọn A:** dùng MCP server demo có sẵn (search hoặc weather)
- **Tùy chọn B:** tự viết một MCP server đơn giản với 1 tool
- **Tùy chọn C:** mock MCP interface với real HTTP call

Điều cần chứng minh

- Tool Worker gọi MCP để lấy dữ liệu
- Kết quả từ MCP được ghi vào shared state
- Trace ghi lại lần gọi MCP

Điều quan trọng nhất

Không quan trọng tool làm gì. Quan trọng là học viên thấy được **cách agent kết nối với capability bên ngoài theo chuẩn**, không phải hard-code.

System pieces

- 1 supervisor với routing logic rõ
- 2-3 workers rõ vai trò
- 1 MCP-connected capability
- Shared state schema có trường trace

Evidence

- trace / logs đọc được
- output cuối cùng kèm source
- ghi chú: route hợp lý chưa? tại sao?
- ít nhất 1 ví dụ về worker fail gracefully

Lưu ý: Không cần build hệ enterprise. Điều cần chứng minh là **việc chia vai trò giúp hệ thống rõ hơn, dễ kiểm soát hơn, và mở rộng tốt hơn.**

Tiêu chí	Đạt (70%)	Tốt (85%)	Xuất sắc (100%)
Phân vai trò	có supervisor + 2 workers	vai trò rõ, không overlap	tránh anti-pattern có ý thức
MCP kết nối	có 1 MCP call hoạt động	schema rõ, trace ghi được	discovery đúng, có error handling
Shared state	state hoạt động	schema đầy đủ, có trace field	ownership rõ từng field
Trace quality	log cơ bản	đủ 5 trường cần thiết	actionable, dẫn tới insight
Routing logic	routing hoạt động	conditional edge rõ	giải thích được quyết định route

Rubric dùng chung cho chấm chéo giữa các nhóm

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 **Multi-agent** là chia vai trò để hệ đỡ quá tải và dễ kiểm soát – chỉ dùng khi bài toán thực sự cần.
- 2 **Supervisor-worker** là pattern practical nhất để bắt đầu: supervisor route, worker chuyên một năng lực hẹp.
- 3 **MCP** nối agent với tool qua chuẩn chung; **A2A** cho agents giao việc cho nhau bằng message contract rõ.
- 4 **LangGraph** biến orchestration thành graph có state và conditional routing – dễ debug, dễ visualize.
- 5 **Observability** là điều kiện tiên quyết: trace tốt là dữ liệu để cải thiện hệ thống lâu dài.

Agent UX & Thiết Kế Trải Nghiệm AI

“Hệ thống đã thông minh hơn và phối hợp tốt hơn. Nhưng nếu trải nghiệm kém, user vẫn sẽ không muốn dùng.”

- Nếu supervisor và workers làm đúng nhưng user không hiểu chuyện gì vừa xảy ra, trải nghiệm có đủ tốt không?
- Quan sát lab hôm nay để nhận ra chỗ nào cần transparency, confidence indicator, và human handoff rõ ràng
- Chuẩn bị 1 use case để mô tả AI flow từ góc nhìn người dùng thay vì chỉ từ kiến trúc

1. Model Context Protocol, *Official Documentation* – modelcontextprotocol.io – client / server model, tools, resources, prompts.
2. LangGraph Docs, *Multi-Agent Tutorials* – supervisor-worker orchestration, graph routing, state handling, human-in-the-loop.
3. Sumers et al. (2023), *Cognitive Architectures for Language Agents (CoALA)* – phân loại memory, action, và decision trong language agents.
4. Anthropic (2024), *Building Effective Agents* – blog post về practical patterns cho agentic systems.
5. LangSmith Documentation, *Tracing & Observability* – công cụ trace cho LangChain / LangGraph pipelines.

Hỏi & Đáp

Một agent rất giỏi có thể làm nhiều việc. Nhưng hệ thống tốt hơn thường bắt đầu từ câu hỏi: nên chia vai trò ở đâu để dễ kiểm soát và dễ cải thiện nhất?